# Lab Assignments
# Computer Graphics DT519A,
# Örebro University, autumn 2024

October 28, 2024

## General instructions

### Deadlines and grading

Refer to *assessment criteria* in a separate document on Blackboard for what is required for grades A–E.

The course schedule lists *nominal deadlines* for each of the lab tasks. These are not hard deadlines, but rather indicate the intended progression. You should be ready with your labs by these dates in order not to fall behind in the course. We grade labs as soon as we can after they are handed in.

**Important!** The first *hard deadline* is at the end of the semester: **January 19**. Solutions handed in before this date will be graded within three weeks. There are two follow-up deadlines: **March 30** and **August 31**. Solutions handed in before each deadline will be graded within three weeks after the deadline. *Solutions handed in after the final deadline will not be graded.* If you miss the last deadline, you will have to register to next instance of the course.

### Pair work

For the labs you should work either individually or in pairs (of 2).

### Plagiarism and cheating

Although you are encouraged to discuss different solutions with your classmates (using the discussion forum on Blackboard is a good idea!) you are *not allowed* to copy code. Copying someone elses code is plagiarism, and will be reported to the university disciplinary board. The same rule applies for copying code from the web, including ChatGPT and other generative AI tools, or from the book.

- If you copy code without specifying the source, you may be charged with cheating.
- If you copy code and attribute the source, but cannot explain what the code does, you mail fail the lab assignment.

### What to report

For each lab, you need to hand in your code (which should run without errors) as well as a document that answers the questions from the lab instructions. The solutions must be submitted via Blackboard.

### Code

Check that your code is correct, can be run without errors and is well formatted and commented before you hand it in.

You are encouraged to use the ORU git server at https://git-e.oru.se. If you do so, please tag the version you wish to submit, and let us know which tag to check out via the Blackboard interface.

### Answers to questions

The instructions list specific questions that need to be answered in your written report. Include a document with your answers, together with the code, when you submit your solutions.

### Presentation/demonstration

After having submitted your code and report, you are required to demonstrate your work to a teacher. All students in a pair should be prepared to answer questions about your implementation, to ensure that all students have learned what is required in the labs.

This demonstration can be done during a lab session if time permits, or otherwise after booking a meeting with a teacher.

## Programming environment

The lab skeletons have been tested with Visual Studio in the lab room T006, and in Ubuntu Linux. Since OpenGL and related libraries are freely available also for most other platforms, you can also use your own laptops and work with the labs on other platforms. However, please note that it is your own responsibility to make compilation and execution work in case you choose alternative platforms. The teachers can help you best if you use Linux or Windows. We don't have much experience with macOS.

## Preparations

In order to benefit properly from the scheduled lab time you are expected to prepare well before each session. Remember that this is for your own benefit. These preparations include:

1. To have read the exercises and any other material such as lab skeletons, or other source files and instructions. If any of the exercise assignments or questions are unclear you should be able to ask the instructor during the scheduled session.

2. Ideally, you should start to work on the labs before the supervised lab session, so that you are ready with questions if you have been getting stuck.

3. Go through the lecture notes and course literature covering the corresponding area.

Additionally, please observe that the lab assignments are not designed to necessarily be completed within the scheduled lab time. You will probably have to work outside the scheduled time in order to finish in time. You are expected to be able to handle the programming environment and to debug your own code.

The programming labs (Lab 1–4) are divided into several tasks (Task 1.1 etc). For each task, read through all of the instructions *before* starting to implement something.

## Debugging tips

Debugging graphics programming can sometimes be frustrating. If your window just turns out black, it can be hard to know what has gone wrong, and it is not generally possible to do printouts from code that runs on the GPU.

One trick that you can try in case you just get a black window is to set `glClearColor` to something other than black, on the CPU side. (For example, try setting it to something like `glClearColor( 0.25f, 0.25f, 0.50f, 1.0f );`) That will let you see if your geometry is where you expect it to be (if you can see some black shape over the coloured background), or if there is just a problem with the *shading* (lights, colours, etc).

Another very useful trick, if the geometry is visible, but there is a problem with the colouring, is to set the colour (the output of the fragment shader, on the GPU side) to a known value. Depending on what you want to find out, you can set it to just a plain value (e.g., `frag_colour = vec3(0.5, 0.8, 1.0);`), or to the normal vector of the surface (`frag_colour = vec3(n.x, n.y, n.z);`), or something else. Just recall that the first component of the output vector will get rendered as a red shade, the second as green, and the third as blue.